LAB 5 Part2 : Binary Tree

[CO2]

Instructions for students:

- Complete the following methods.
- You may use Java / Python to complete the tasks.
- DO NOT CREATE a separate folder for each task just follow the given template.
- If you are using **JAVA**, then follow the **Java Template**.
- If you are using **PYTHON**, then follow the **Python Template**.

NOTE:

- **YOU CANNOT USE ANY OTHER DATA STRUCTURE OTHER THAN ARRAY UNLESS MENTIONED IN THE QUESTION.**
- **YOUR CODE SHOULD WORK FOR ANY VALID INPUTS.**

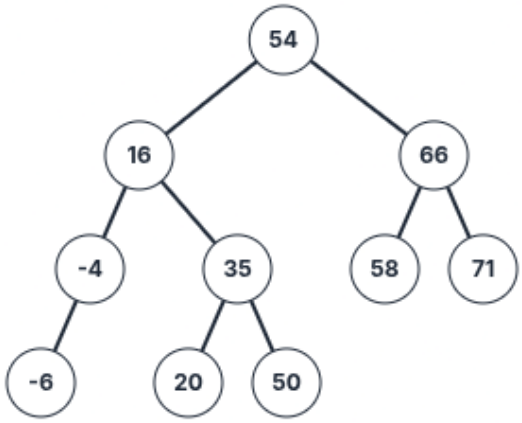
For Python: List, Negative indexing and append()
and
For Java: Static or Instance variables
are STRICTLY prohibited

Lab Tasks: 3 tasks (1~3)
Assignment Tasks: 3 tasks (4~6)
Total Assignment Mark: 3*5=15

LAB TASKS (no need to submit)

1. Binary Search Tree Minimum and Predecessor:

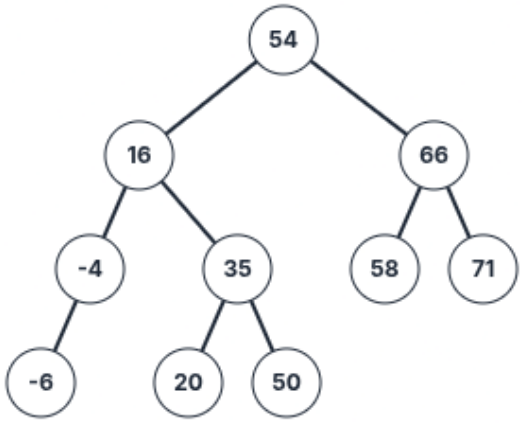
a. Find the maximum number from a Binary Search Tree



Sample Call	Result
`getMaximum(root)`	71

Note: You have to keep in mind that, the question says it's a Binary Search Tree (not a regular BT). Therefore, you must use BST logic to find the maximum.

b. Find a InOrder Predecessor from a Binary Search Tree



Sample Call	Result
`inOrderPred(root,54)`	50
`inOrderPred(root,66)`	58
`inOrderPred(root,20)`	16
`inOrderPred(root,50)`	35
`inOrderPred(root,-6)`	null/None

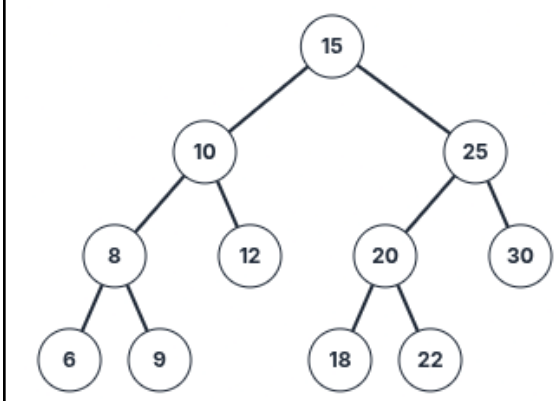
Note: You should reuse the lab task 1a to solve 1b.

You can solve 1a and 1b without using any recursion. Can you guess why we can solve them without recursion even though they are binary tree problems?

2. Find the Lowest Common Ancestor (LCA) of two nodes in a BST

The lowest common ancestor (LCA) of two nodes x and y in the BST is the lowest (i.e., deepest) node that has both x and y as descendants. In other words, the LCA of x and y is the shared ancestor of x and y that is located farthest from the root.

Corner Case: if y lies in the subtree rooted at node x, then x is the LCA; otherwise, if x lies in the subtree rooted at node y, then y is the LCA.



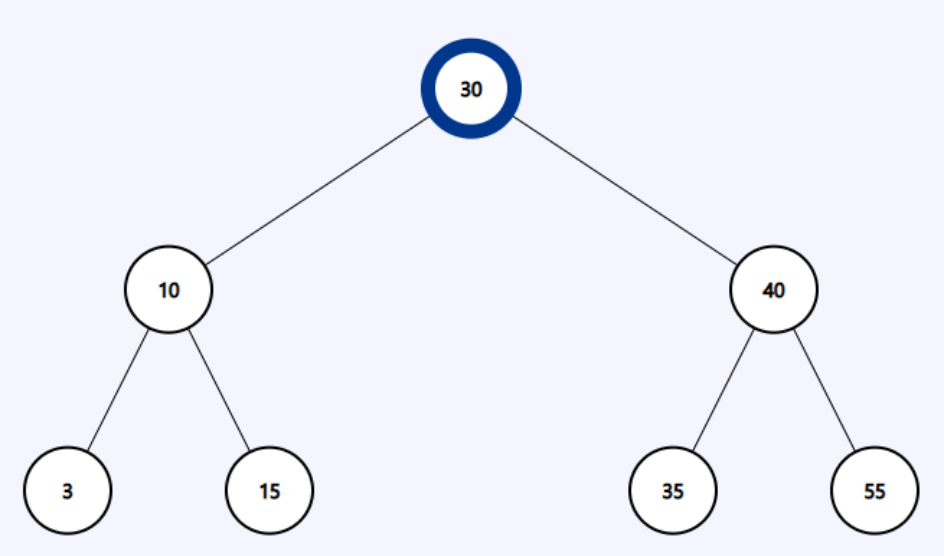
Sample Call	Result
LCA(root, 6,12)	10
LCA(root, 20,6)	15
LCA(root, 18,22)	20
LCA(root, 20,25)	25
LCA(root, 10,12)	10

3. Find the path

Faria studies in Rangpur Medical College. During her Eid vacation, she is planning to visit her family and relatives in Chittagong. But she feels really bad during the journey. As a result, you need to help Faria reach her destination.

A Binary Search Tree is given.The root node is the starting position of Faria and a key node will be given as the destination. Now, you have to find if the following key node (which is the destination of Faria) is present in the tree or not. If present, return the path from the root node to the key node else print “No Path Found”.

[You can use Python List for this task]

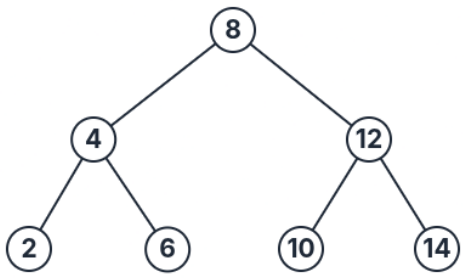
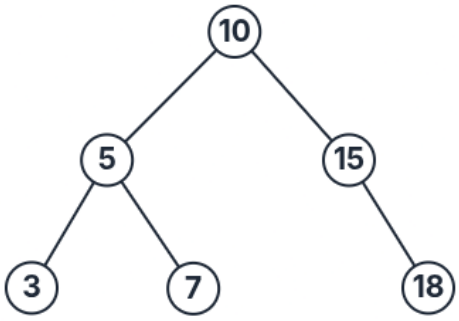


Sample Input	Sample Output
Source node(root): 30 Destination node(key): 15	Path: [30, 10, 15]
Source node(root): 30 Destination node(key): 50	No Path Found

ASSIGNMENT TASKS (must submit)

4. Sum of range

Write a function **rangeSum(root,low,high)** that takes the root of a Binary Search Tree as a parameter and sums up the values within the range mentioned in the parameter.

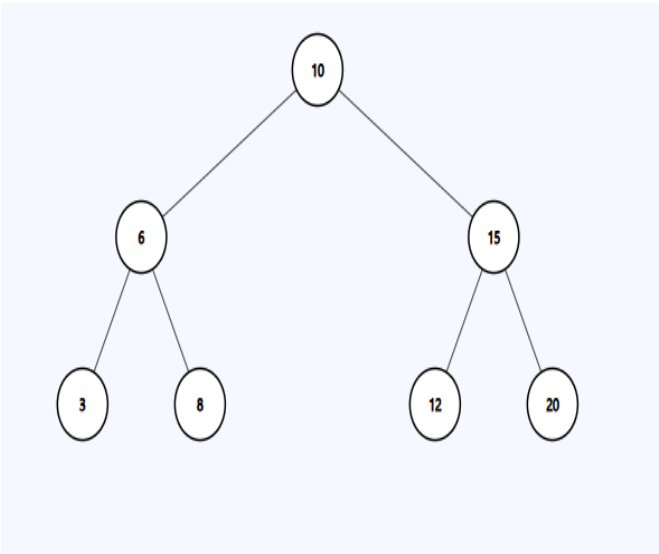
Sample Input	Sample Function Call	Sample Returned Value
	`rangeSum(root, 5, 13)`	**36** **Explanation:** 6+8+10+12=36
	`rangeSum(root, 7, 15)`	**32** **Explanation:** 7+10+15=32

5. Mirror Sum of a BST

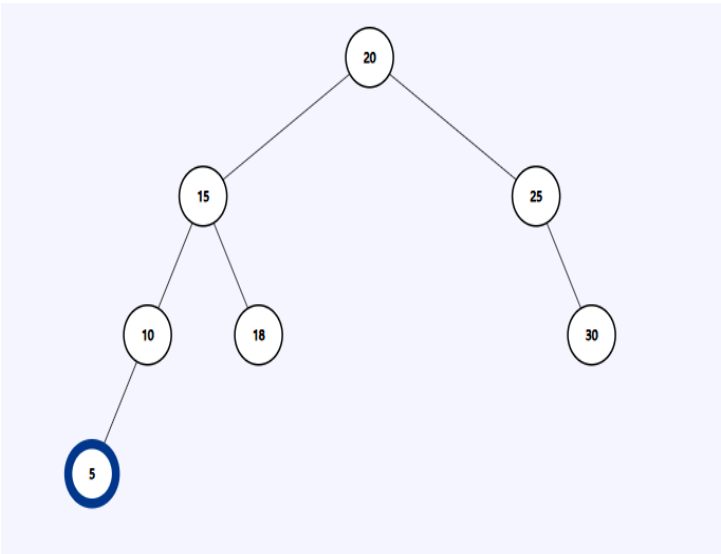
You are given the root node of a Binary Search Tree. You need to calculate the **sum** of the values of the nodes that are **mirrors** of each other then **return** it. Here, mirror means the nodes that are located in corresponding positions in the left and right subtrees of the root. If the mirror doesn't exist then do not sum.

Note: You can use helper functions.

Example Tree input 1



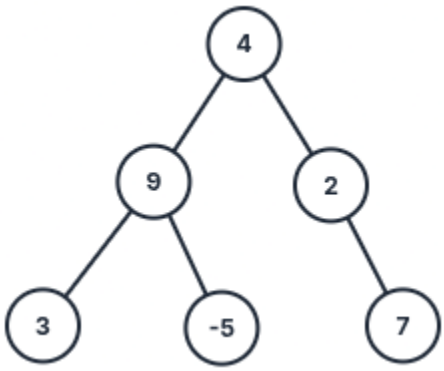
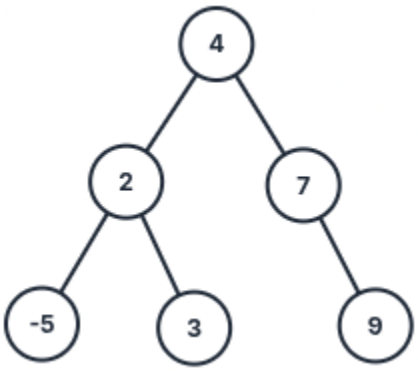
Example Tree input 2



Sample Input 1	Sample Output 1	Explanation 1
mirrorSum(root)	64	For Tree 1 Mirror nodes are: 6 and 15, sum = 6 + 15 = 21 3 and 20, sum = 3 + 20 = 23 8 and 12, sum = 8 + 12 = 20 Total Mirror Node Sum = 21+23+20 = 64
Sample Input 2	**Sample Output 2**	**Explanation 2**
mirrorSum(root)	80	For Tree 2 Mirror nodes are: 15 and 25, sum = 15 + 25 = 40 10 and 30, sum = 10 +30 = 40 Total Mirror Node Sum = 40 + 40 = 80

6. Check BST

Write a function **isBST(root)** that takes the root of a Binary Tree as a parameter and then returns true if its a BST otherwise returns false.

Sample Input	Sample Returned Value
	**false**
	**true**